Bioinformatics, 2026, pp. 1–4

Application Note

AIRRPrep: Workflows for Preparing Bulk and Single-Cell AIRR-Seq Data for Annotation

Parsa Esmaeili,[1,†] Fatemeh Chaker Hosseini Zavareh,[1,†] Nika Abdollahi[1,2] and Changiz Eslahchi[1,*]

[1]ComputImm Lab, School of Biological Sciences, Institute for Research in Fundamental Sciences (IPM), Farmanieh Campus, 19395-5746, Tehran, Iran

[2]IMGT®, the international ImMunoGeneTics Information System®, Institute of Human Genetics, University of Montpellier, Scientific Research National Center (CNRS), Montpellier, France

[†]These authors contributed equally and are co-first authors.

[*]Corresponding author. cheslahchi@computimm.com

Abstract

Summary: AIRRPrep is a web application for reproducible preprocessing of bulk and single-cell AIRR-seq data, providing predefined and customizable pRESTO workflows with input-aware compatibility checks and stepwise provenance, alongside harmonization of seven single-cell input specifications with linked retention of available source annotations. Three predefined bulk workflows showed record-level agreement with equivalent command-line pRESTO workflows at every compared step. For the tested single-cell adapters, harmonized outputs preserved the evaluated sequence-to-cell links and available metadata.

Availability and Implementation: AIRRPrep is freely available without registration at `https://preprocessing.computimm.com`. The Python backend, React frontend, container definitions and validation resources are available under the GNU Affero General Public License v3.0 or later at `https://github.com/ComputImm/AIRRPrep`. Release `v1.0.0` is archived at `https://doi.org/10.5281/zenodo.22857985`.

Contact: cheslahchi@computimm.com

Supplementary information: Supplementary data are available at *Bioinformatics* online.

Keywords AIRR-seq, immune repertoire, preprocessing, pRESTO, TRUST4

Introduction

Adaptive immune receptor repertoire sequencing (AIRR-seq) data enter analysis as bulk reads or heterogeneous single-cell inputs. Here, preprocessing refers to preparing receptor sequences and associated metadata for subsequent V(D)J annotation or reannotation and repertoire analysis. For bulk inputs, this involves configurable read-processing operations. For single-cell inputs, it involves receptor reconstruction when required and format and metadata harmonization (hereafter, harmonization) of sequences, cell identifiers and available metadata across supported input formats. Inputs arrive at different processing stages and carry different information, requiring preparation appropriate to their content and structure.

Bulk reads may require quality filtering, primer masking, unique molecular identifier (UMI) extraction, paired-read assembly, consensus generation and duplicate collapsing before V(D)J assignment. These operations are order- and state-dependent: file formats, available annotations, read-stream relationships and required companion files determine which subsequent operations are computationally valid. Single-cell inputs may instead comprise raw reads, reconstructed sequences or platform-specific contig and annotation files. Their harmonization must preserve available sequence-to-cell links and source annotations without inventing missing identifiers. Incompatible workflow steps or lost metadata links can compromise downstream analysis.

Existing software addresses these tasks at different levels. pRESTO provides command-line preprocessing of raw AIRR-seq reads (Vander Heiden et al., 2014). TRUST4 reconstructs receptor sequences from bulk and single-cell RNA-seq (Song et al., 2021), whereas nf-core/airrflow provides a configuration-driven, end-to-end workflow for both bulk and single-cell AIRR-seq (Gabernet et al., 2024). Web platforms provide overlapping preprocessing, annotation and repertoire-analysis capabilities: VDJServer extends from read preprocessing and V(D)J gene assignment to repertoire analysis and data sharing (Christley et al., 2018), while ARGalaxy supports read trimming and demultiplexing, IgBLAST-based annotation, and downstream repertoire analysis and visualization (IJspeert et al., 2017). For single-cell receptor data, scirpy supports import from Cell Ranger, TraCeR and AIRR-formatted files for downstream analysis and integration with transcriptomic data (Sturm et al., 2020).

These tools provide complementary capabilities. Direct use of pRESTO requires users to maintain compatible file and header-annotation states and to order interdependent operations correctly. AIRRPrep addresses these workflow-construction responsibilities through input-aware checks and combines this functionality with preparation of heterogeneous single-cell inputs for downstream

© The Author(s) 2026. Copyright and licence statements to be updated by the publisher during production.

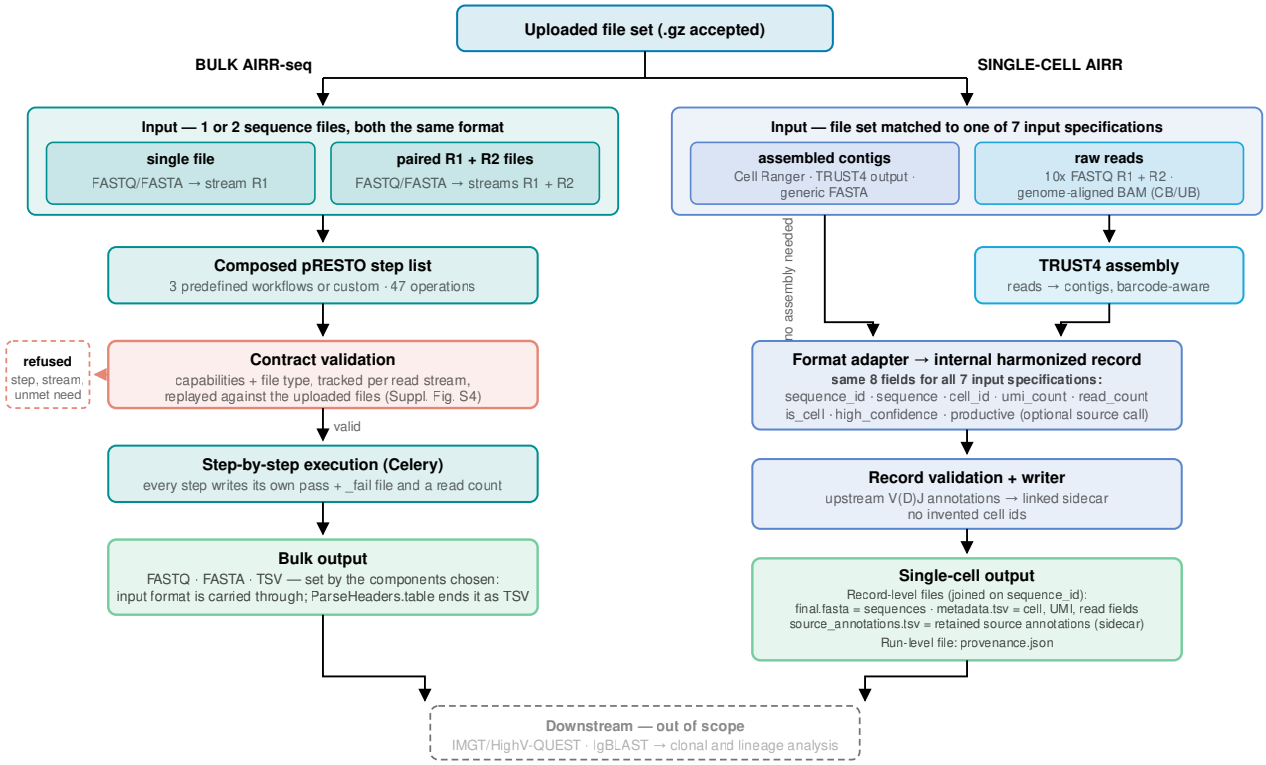


Figure 1 Overview of AIRRPrep. Bulk inputs are processed using predefined or custom pRESTO workflows whose computational compatibility is checked before execution. Single-cell inputs are harmonized directly when reconstructed sequences are provided or are first assembled with TRUST4 when assembly is required. Available source annotations are retained in a linked sidecar keyed by the same `sequence_id` as the harmonized output.

annotation or reannotation. Its contribution lies in this combination of workflow validation, sequence and metadata harmonization, and linked retention of available source annotations, rather than heterogeneous single-cell import alone. Supplementary Table S11 summarizes the capabilities assessed in related tools; features not evaluated are distinguished from those documented as absent.

For bulk data, AIRRPrep offers three predefined protocols and a custom builder covering 47 pRESTO operations. State-aware contracts track file type, available annotations, companion-file requirements and R1/R2 state during construction and before execution, identifying computationally incompatible steps before they run. For single-cell data, AIRRPrep accepts reconstructed inputs or invokes TRUST4 when assembly is required, then maps seven supported input specifications to a common sequence-and-metadata representation. It does not invent missing cell identifiers and retains available source annotations in a linked sidecar keyed by the same `sequence_id`. Workflow graphs, per-step record counts, provenance and exportable definitions support inspection and rerunning. These checks establish computational coherence, not the biological appropriateness of parameter choices.

Implementation

AIRRPrep provides distinct processing routes for bulk and single-cell AIRR-seq data, as summarized in Figure 1.

Bulk workflow construction and pre-execution validation

For bulk processing, AIRRPrep represents input as either one read stream from a FASTA or FASTQ file or separate R1 and R2 streams from matched paired-end files of the same format. Three predefined workflows reproduce documented pRESTO examples: UMI-barcoded MiSeq 2×250 (Stern et al., 2014), non-UMI MiSeq 2×250 (Greiff et al., 2014) and UMI-barcoded 5′ RACE MiSeq 325+275 (Vander Heiden et al., 2017); full parameters are listed in Supplementary Table S9. A custom builder lets users compose workflows from 47 operations across 13 pRESTO tools (Supplementary Table S2). Each operation declares a contract specifying accepted file types, required and produced capabilities, companion files and read-stream behavior (Supplementary Figure S4). Whereas pRESTO validates individual commands at execution, AIRRPrep propagates file type, available annotations and R1/R2 state through the proposed workflow beforehand. It rejects the first incompatible step and reports the affected step, stream and unmet requirement, such as quality filtering on FASTA or consensus generation before barcode annotation. The same contracts update which operations are selectable during construction and explain why others are disabled, guiding users toward workflows that are computationally coherent but not necessarily biologically appropriate (Supplementary Note S1). The interface also displays a count-annotated workflow graph, tracking R1 and R2 separately until assembly and the merged stream thereafter; the graph is exportable as a PNG (Supplementary Figures S2 and S3).

Single-cell format and metadata harmonization

Single-cell repertoire inputs contain receptor sequences together with cell-associated information, including cell barcodes, UMI counts and read support. AIRRPrep preserves available sequence-to-cell relationships while converting heterogeneous input formats into a unified representation for downstream repertoire analysis. Supported inputs include reconstructed outputs from Cell Ranger and TRUST4, as well as raw 10x FASTQ and genome-aligned BAM files (Supplementary Table S1). When reconstructed sequences are provided, AIRRPrep directly parses and harmonizes the existing output. For raw sequencing inputs, receptor assembly is first performed with TRUST4, followed by parsing and harmonization of the resulting sequences and associated cell-level information.

Format-specific adapters convert supported inputs into AIRRPrep's eight-field internal record, which is not an AIRR Rearrangement record. The run writes three record-level outputs: `final.fasta`, which carries the receptor sequences; `metadata.tsv`, which carries the available cell identifiers, UMI counts and read support; and `source_annotations.tsv`, the sidecar holding the retained source annotations. One run-level file, `provenance.json`, completes the output. Absent optional metadata are left as empty fields rather than inferred. The sequence-to-cell link is treated differently: cell identifiers are never generated from sequence identifiers alone, inputs lacking a valid cell mapping are refused, and TRUST4 contigs that the source assigns to no cell are excluded and counted (Supplementary Table S1). The optional `productive` field is copied from Cell Ranger only on request; it remains a source call and is not inferred by AIRRPrep. This approach maintains available links between receptor sequences and their originating cells while providing a consistent structure across input sources.

Upstream receptor annotations, including V(D)J gene calls, junctions and CDR3 annotations, are excluded from the harmonized representation to enable consistent downstream annotation using a defined reference and procedure, such as IMGT/HighV-QUEST (Alamyar et al., 2012). Available source annotations are retained separately in `source_annotations.tsv`, linked to output records through the same `sequence_id`. Processing information and checksums for the whole run are recorded in `provenance.json` (Supplementary Table S1).

Reproducibility and operation

Workflow definitions can be exported for reuse, while completed runs provide exportable records of workflow structure, outputs and execution metadata. Workflow graphs can be exported as images, allowing users to document the executed pipeline in manuscripts or reports. External companion files, such as primer FASTA inputs, are not embedded in workflow definitions; instead, required inputs are recorded with their file names, sizes and SHA-256 checksums. Exported records also include the software versions used during execution. Long-running jobs can be recovered using email-based tracking without requiring a persistent user account (Supplementary Note S2).

AIRRPrep uses a web-based client-server architecture with asynchronous job execution and state management through worker processes and Redis. The containerized environment includes TRUST4 (Song et al., 2021), BLAST+ (Camacho et al., 2009), samtools (Danecek et al., 2021), MUSCLE (Edgar, 2004), VSEARCH (Rognes et al., 2016) and CD-HIT (Fu et al., 2012) and can be deployed on institutional infrastructure. The proprietary USEARCH is not included, and no predefined workflow depends on it. Software versions and execution dependencies are listed in Supplementary Table S10.

Validation

The three predefined bulk workflows were evaluated against equivalent command-line pRESTO implementations using the first 25,000 read pairs from SRR4026043 (Vander Heiden et al., 2017), ERR346600 (Greiff et al., 2014) and SRR1383456 (Stern et al., 2014). Across all three workflows, AIRRPrep and pRESTO produced identical records, sequences, quality strings and annotations at each compared step; only record ordering differed (Supplementary Tables S3–S5). Of the 47 operations, 46 produced output equivalent to native pRESTO in one archived test case each. `ConvertHeaders.py migec` could not be compared because native pRESTO fails on multi-record MIGEC input. In a separate USEARCH-specific `ClusterSets` test, native pRESTO's generated command failed, whereas AIRRPrep's corrected invocation completed successfully. The workflow validator accepted the predefined workflows and rejected invalid compositions involving missing annotations, incompatible file types, missing companion files and invalid read-stream states (Supplementary Note S1).

Single-cell harmonization was evaluated using Cell Ranger representations of the 10x Genomics `sc5p_v2_hs_PBMC_10k` T-cell V(D)J dataset. Harmonized outputs matched the reference records without sequence, cell barcode, UMI, read count or cell-status discrepancies (Supplementary Table S6). Generic FASTA inputs reproduced the same records when barcode mappings were provided, while inputs lacking a valid barcode mapping were rejected. TRUST4-based routes were evaluated using raw 10x reads, a contig-aligned BAM and a genome-aligned gene-expression BAM from the `sc5p_v2_hs_PBMC_1k` dataset. The raw-read and contig-aligned BAM routes recovered all 83 Cell Ranger-defined cells, whereas the genome-aligned gene-expression BAM route recovered 79 of 83; no route assigned artificial cell identifiers (Supplementary Table S7).

Execution time was evaluated on three replicates using identical pRESTO and Python environments on a Windows 11 workstation. AIRRPrep reduced execution time for the tested 25,000-read-pair workflows, primarily by avoiding repeated interpreter startup overhead between steps. Reference-guided assembly showed similar runtime between implementations because the underlying alignment operation dominated execution time. These measurements represent small-scale comparative execution tests on one workstation and are not intended as estimates of production-scale throughput.

Discussion

AIRRPrep simplifies access to established AIRR-seq preprocessing workflows while adding machine-checkable safeguards around workflow composition during construction and before execution. Stepwise record tracking makes runs auditable, and exported workflow definitions support reproducibility. Validation demonstrates computational coherence rather than biological appropriateness: a technically valid workflow can still contain unsuitable choices, such as an incorrect primer set or offset.

Read-pair assembly dominated the tested bulk workflows; TRUST4-based single-cell reconstruction was not included in the runtime comparison. AIRRPrep is intentionally focused on preprocessing:

it does not provide a separate downstream V(D)J reannotation stage, and its single-cell output is an internal representation rather than an AIRR Rearrangement record (Vander Heiden et al., 2018); therefore, AIRR compliance is not claimed. Future work will include cross-step quality-control summaries, support for additional single-cell platforms, resource-matched Linux benchmarks at production input sizes, and integration with downstream annotation tools.

Data availability

AIRRPrep is available as a free hosted service at `https://preprocessing.computimm.com`. The frontend and backend source code, container definitions, automated tests, validation scripts and their archived outputs (Supplementary Tables S2–S7), together with benchmark scripts and results (Supplementary Tables S8–S9), are available at `https://github.com/ComputImm/AIRRPrep` under the GNU Affero General Public License v3.0 or later. The version described in this manuscript is release `v1.0.0` (commit `9148dba8bdd4c516b80db4eece07dd0a0b23a929`), archived at `https://doi.org/10.5281/zenodo.22857985`.

Large validation and benchmark outputs that exceed repository limits, including per-operation validation outputs, single-cell validation outputs and benchmark run directories, are archived separately at `https://doi.org/10.5281/zenodo.22858375`. SHA-256 checksums are provided for archived files, and the repository links supplementary tables to the scripts and generated outputs used to produce them.

The bulk validation datasets consist of the first 25,000 read pairs from SRR4026043 (Vander Heiden et al., 2017), ERR346600 (Greiff et al., 2014) and SRR1383456 (Stern et al., 2014). Single-cell validation used Cell Ranger 5.0.0 outputs from the 10x Genomics `sc5p_v2_hs_PBMC_10k` and `sc5p_v2_hs_PBMC_1k` datasets. Input file checksums and validation outputs are provided in Supplementary Tables S6 and S7. USEARCH is proprietary and is not redistributed; the corresponding commands, software version and test results are provided for users with an appropriate licence.

Author contributions

Parsa Esmaeili: Conceptualization, Software, Methodology, Validation, Data curation, Writing – original draft, Writing – review and editing. Fatemeh Chaker Hosseini Zavareh: Conceptualization, Software, Methodology, Validation, Data curation, Writing – review and editing. Nika Abdollahi: Conceptualization, Methodology, Resources, Validation, Writing – review and editing. Changiz Eslahchi: Conceptualization, Software, Supervision, Writing – review and editing.

Funding

No specific funding was received for this work.

Conflict of interest

None declared.

References

Vander Heiden JA, Yaari G, Uduman M, et al. pRESTO: a toolkit for processing high-throughput sequencing raw reads of lymphocyte receptor repertoires. *Bioinformatics*. 2014;30(13):1930–1932.

Song L, Cohen D, Ouyang Z, Cao Y, Hu X, Liu XS. TRUST4: immune repertoire reconstruction from bulk and single-cell RNA-seq data. *Nature Methods*. 2021;18:627–630.

Gabernet G, Marquez S, Bjornson R, et al. nf-core/airrflow: an adaptive immune receptor repertoire analysis workflow employing the Immcantation framework. *PLoS Computational Biology*. 2024;20(7):e1012265.

Alamyar E, Duroux P, Lefranc M-P, Giudicelli V. IMGT/HighV-QUEST: the IMGT web portal for immunoglobulin and T cell receptor NGS repertoire analysis. *Immunome Research*. 2012;8:26.

Christley S, Scarborough W, Salinas E, et al. VDJServer: a cloud-based analysis portal and data commons for immune repertoire sequences and rearrangements. *Frontiers in Immunology*. 2018;9:976.

IJspeert H, van Schouwenburg PA, van Zessen D, Pico-Knijnenburg I, Stubbs AP, van der Burg M. Antigen receptor Galaxy: a user-friendly, web-based tool for analysis and visualization of T and B cell receptor repertoire data. *The Journal of Immunology*. 2017;198(10):4156–4165.

Vander Heiden JA, Marquez S, Marthandan N, et al. AIRR Community standardized representations for annotated immune repertoire data. *Frontiers in Immunology*. 2018;9:2206.

Greiff V, Menzel U, Haessler U, et al. Quantitative assessment of the robustness of next-generation sequencing of antibody variable gene repertoires from immunized mice. *BMC Immunology*. 2014;15:40.

Stern JNH, Yaari G, Vander Heiden JA, et al. B cells populating the multiple sclerosis brain mature in the draining cervical lymph nodes. *Science Translational Medicine*. 2014;6(248):248ra107.

Vander Heiden JA, Stathopoulos P, Zhou JQ, et al. Dysregulation of B cell repertoire formation in myasthenia gravis patients revealed through deep sequencing. *The Journal of Immunology*. 2017;198(4):1460–1473.

Camacho C, Coulouris G, Avagyan V, et al. BLAST+: architecture and applications. *BMC Bioinformatics*. 2009;10:421.

Danecek P, Bonfield JK, Liddle J, et al. Twelve years of SAMtools and BCFtools. *GigaScience*. 2021;10(2):giab008.

Edgar RC. MUSCLE: multiple sequence alignment with high accuracy and high throughput. *Nucleic Acids Research*. 2004;32(5):1792–1797.

Rognes T, Flouri T, Nichols B, Quince C, Mahé F. VSEARCH: a versatile open source tool for metagenomics. *PeerJ*. 2016;4:e2584.

Fu L, Niu B, Zhu Z, Wu S, Li W. CD-HIT: accelerated for clustering the next-generation sequencing data. *Bioinformatics*. 2012;28(23):3150–3152.

Sturm G, Szabo T, Fotakis G, et al. Scirpy: a Scanpy extension for analyzing single-cell T-cell receptor-sequencing data. *Bioinformatics*. 2020;36(18):4817–4818.